Тема: Створення API за допомогою Node.js та Express: Частина 1. Створення сервера

Мета: розглянути та практично засвоїти процес створення серверу за допомогою вбудованого модуля Node HTTP та Express, призначення маршрутів для запитів, виконуваних за різними URL-адресами та отримання у запитах за методом GET дані у форматі JSON.

Порядок виконання роботи:

1. Перевірено встановлені версії середовища виконання Node.js та пакетного менеджера npm за допомогою виконання команди node -v; npm -v у терміналі Visual Studio Code (рис. 1).

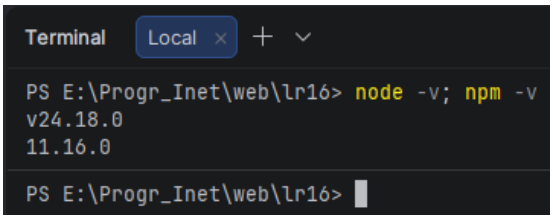


Рисунок 1 – Перевірка встановлених версій Node.js та npm

2. Створено папку проєкту express-api, здійснено перехід до неї та виконано ініціалізацію нового проєкту за допомогою утиліти npm init, у процесі якої задано параметри та автора Humeniuk Denys (рис. 2).

```
PS E:\Progr_Inet\web\lr16> cd express-api

PS E:\Progr_Inet\web\lr16\express-api> npm init
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See `npm help init` for definitive documentation on these fields and exactly what they do.

Use `npm install <pkg>` afterwards to install a package and save it as a dependency in the package.json file.

Press ^C at any time to quit.
package name: (express-api)
version: (1.0.0)
description: Node.js and Express REST API
entry point: (index.js)
test command:
git repository:
keywords: node.js, express, rest, api
author: Humeniuk Denys
license: (ISC)
type: (commonjs)
About to write to E:\Progr_Inet\web\lr16\express-api\package.json:

{
  "name": "express-api",
  "version": "1.0.0",
  "description": "Node.js and Express REST API",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [
    "node.js",
    "express",
    "rest",
    "api"
  ],
  "author": "Humeniuk Denys",
  "license": "ISC",
  "type": "commonjs"
}


Is this OK? (yes)

PS E:\Progr_Inet\web\lr16\express-api>
```

Рисунок 2 – Ініціалізація проєкту командою npm init

3. Після завершення ініціалізації згенеровано первинний конфігураційний файл package.json, який містить базові налаштування проєкту та дані автора (рис. 3).

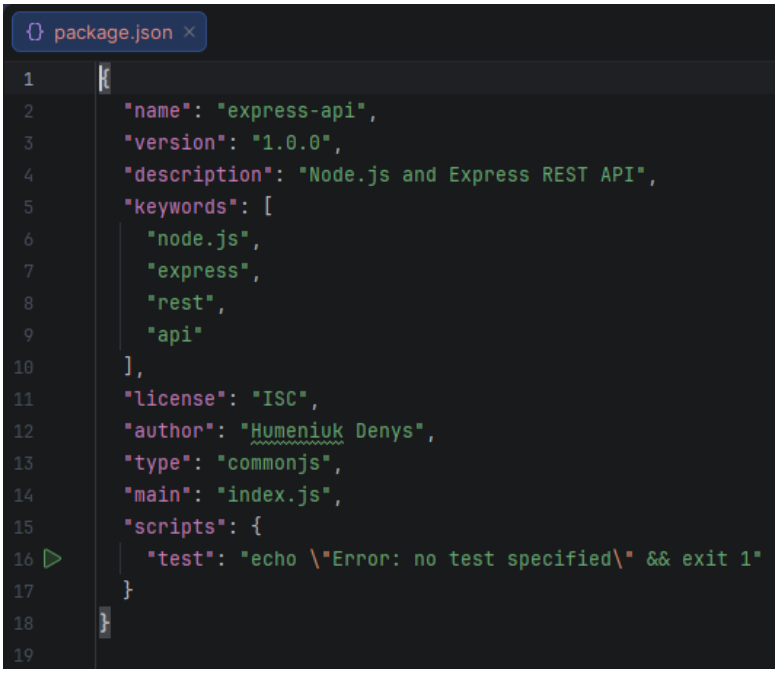


Рисунок 3 – Вміст згенерованого файлу package.json

4. За допомогою команди npm install body-parser express mysql request встановлено необхідні бібліотеки-залежності для розробки REST API (рис. 4).



Рисунок 4 – Встановлення бібліотек-залежностей у проєкт

5. Створено файл hello-server.js та реалізовано базовий HTTP-сервер засобами вбудованого модуля http Node.js на порту 3001 (рис. 5).



Рисунок 5 – Програмний код HTTP-сервера у файлі hello-server.js

6. Сервер запущено командою node hello-server.js та перевірено початок прослуховування порту 3001 у терміналі (рис. 6).

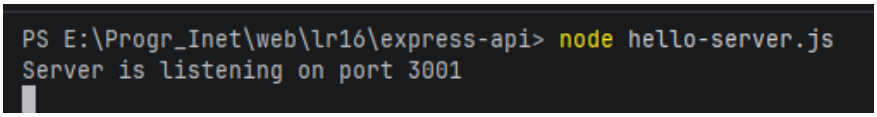


Рисунок 6 – Запуск сервера hello-server.js у терміналі

7. Перевірено доступність сервера у веббраузері за адресою http://localhost:3001/. Отримане повідомлення Hello, server! підтверджує його правильну роботу (рис. 7).

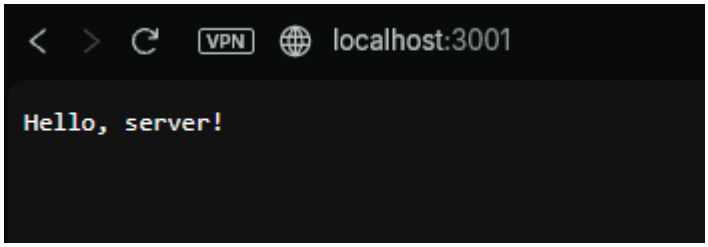


Рисунок 7 – Результат перевірки роботи сервера у браузері

8. Виконано тестування сервера за допомогою утиліти cURL з прапорцем -i для перегляду повернених HTTP-заголовків та тіла відповіді (рис. 8).

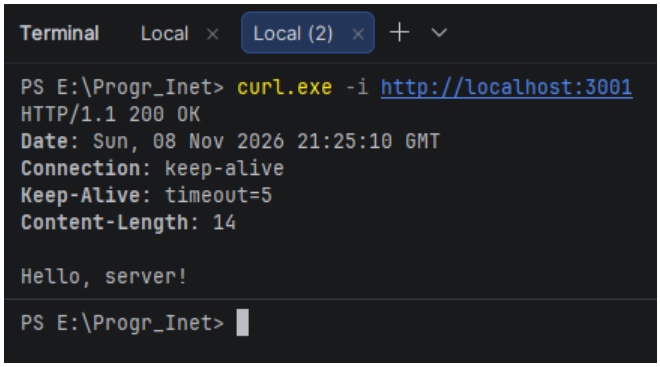


Рисунок 8 – Перевірка сервера за допомогою команди curl.exe -i

9. Створено файл app.js, що є точкою входу для сервера на базі вебфреймворку Express на порту 3002 (рис. 9).

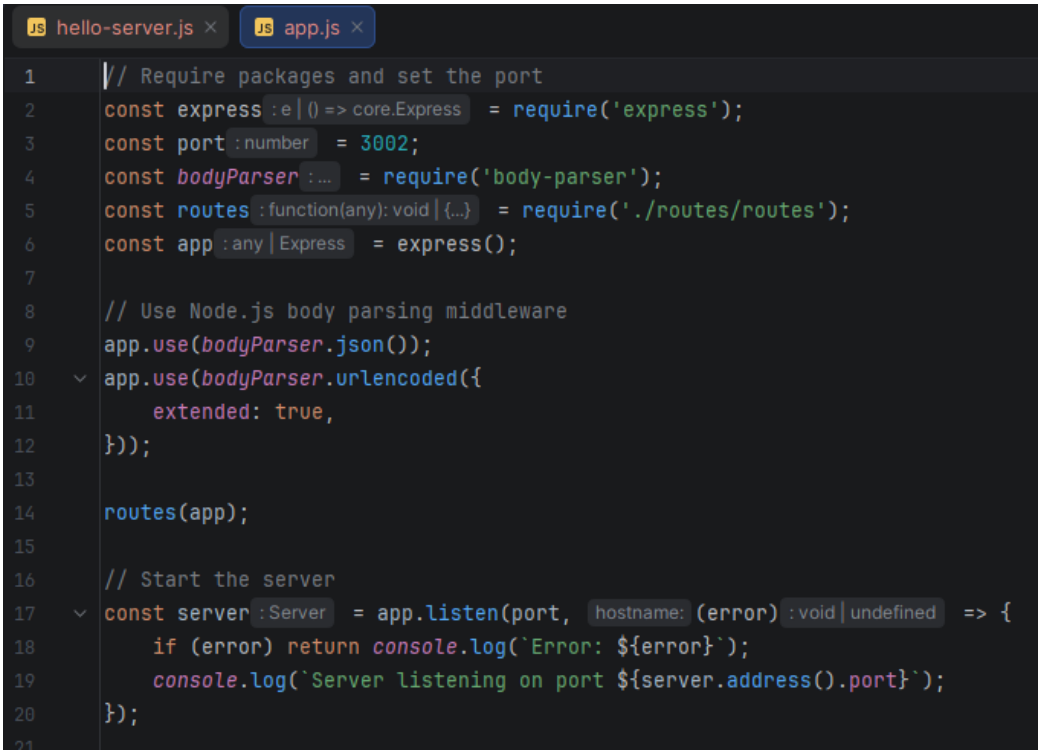


Рисунок 9 – Програмний код сервера Express у файлі app.js

10. У файлі package.json додано команду автоматичного запуску "start": "node app.js", після чого сервер Express успішно запущено командою npm start (рис. 10).



Рисунок 10 – Запуск Express-сервера через npm start

11. За допомогою утиліти cURL перевірено роботу Express-сервера та зафіксовано заголовок Content-Type: application/json; charset=utf-8 при поверненні JSON-відповіді (рис. 11).

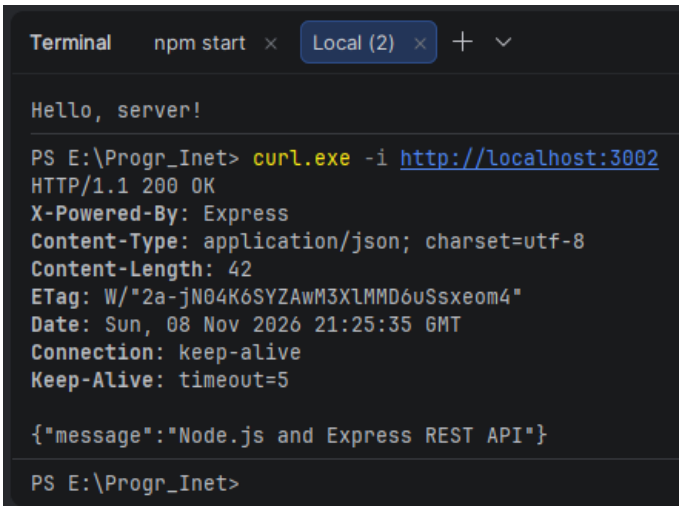


Рисунок 11 – Перевірка заголовків та JSON-відповіді сервера через cURL

12. Перевірено коректне відображення повернених даних у форматі JSON у браузері за адресою http://localhost:3002/ (рис. 12).

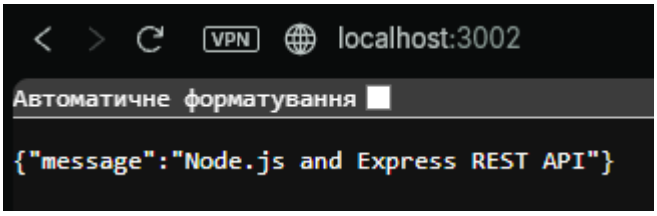


Рисунок 12 – Відображення JSON-відповіді кореневого маршруту у браузері

13. Створено модуль routes/routes.js, до якого винесено логіку маршрутизації, додано масив users з тестовими даними користувачів (Humeniuk Denys, Haliuk Ivan) та обробник маршруту /users за методом GET (рис. 13).

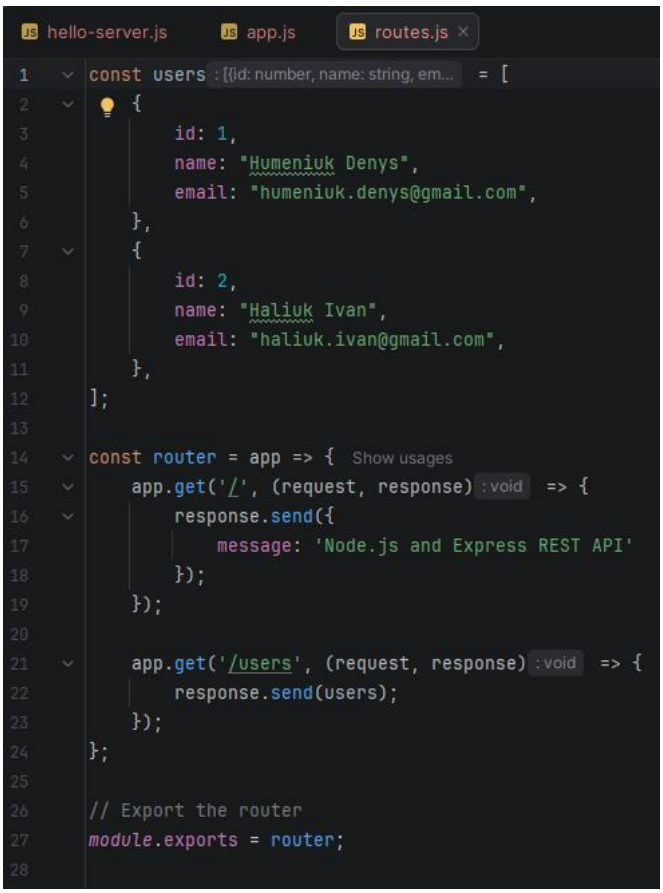


Рисунок 13 – Модуль routes.js із масивом користувачів та обробником /users

14. Перезапущено сервер та перевірено отримання списку користувачів у форматі JSON за адресою http://localhost:3002/users у браузері (рис. 14).

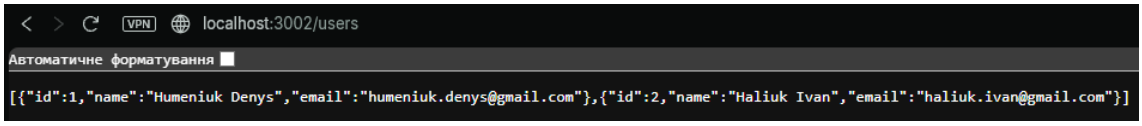


Рисунок 14 – Отримання списку користувачів у браузері за адресою localhost:3002/users

Висновки: на даній практичній роботі розглянуто, як створити сервер за допомогою вбудованого модуля Node HTTP та за допомогою Express, як призначити маршрути для запитів, виконуваних за різними URL-адресами, і як отримати у запитах за методом GET дані у форматі JSON.